

Utility aware Energy Constrained Real-Time Scheduling

Michael Shell, *Member, IEEE*, John Doe, *Fellow, OSA*, and Jane Doe, *Life Fellow, IEEE*

Abstract—Utility aware Energy Constrained Real-Time Scheduling.

Index Terms—Real-Time Systems, Automotive Systems, Conditional DAG.

I. PROBLEMS TO SOLVE

- Offline
 - Job to Processor, Optional Segment, Frequency mapping
 - * EDF feasible mapping at f_{max} for mandatory segment
 - * Heur - To begin, load balanced mapping. So that later optional segments can be scheduled for all tasks. Later, may consider other mapping approaches. OPT will compute it automatically.
 - Maximum optional execution possible under worst-case assumption to maximize utility (or minimize utility loss)
- Online
 - At job completion, time slack and energy slack
 - Time slack - per processor. Energy slack - global.
 - Time Slack - non-accumulative, use it now or in near future, because of deadlines, may not be forwarded
 - Energy slack - accumulative
 - Time slack - (1) when to use - now or later, (2) if used now - (i) use to reduce frequency, increases energy slack, to be used later, (ii) execute additional optional segment - this uses energy slack as well, while energy constraint is satisfied.
 - Energy slack - (1) where to use, (2) when to use - now or later, (3) if used now - (i) increase freq - more time slack for later, (ii) use it with time slack to execute additional optional segment.

1) Approaches:

- **ILP v1.** Pure Discrete Optimization - Optimal ILP
- **ILP v2.** SPS (for mapping) + ILP (for frequency and optional segment)
- Heuristic
- Online

II. SYSTEM MODEL AND PROBLEM FORMULATION

Problem Mapping and Scheduling of segmented real-time tasks. Energy budget. Maximize utility. Mandatory segment always executed. Periodic - **harmonic periods**. Preemptive. **Mapping job level**. EDF.

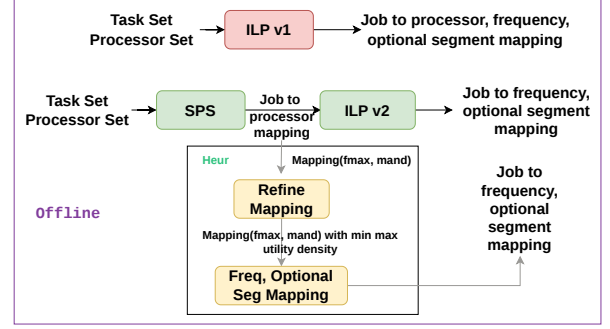


Fig. 1. Approaches

Processor Model. Given a set $\mathcal{P}$ of N^{prc} **homogeneous** processors such that each processor P_x executes with a discrete set of N^{freq} frequency $\mathcal{F} = \{f_z\}$ such that f_z is normalized with respect to the $f^{max} = f_{N^{freq}} = 1$ in the system. Furthermore, frequency values in $\mathcal{F}$ are assumed to be arranged in an increasing order.

Task Model. Given a set $\mathcal{T} = \{T_i\}$ of N^{tsk} real-time periodic tasks - **harmonic periods**. Each task T_i is specified with the following parameters.

- Each task is activated periodically with a period of p_i , each such activation is referred to as the *job* of this task.
- j^{th} job of T_i is activated at $r_{i,j} = (j - 1) \times p_i$ with a *deadline* $d_{i,j} = j \times p_i$
- Each task T_i (and so does each job of this task) has a *mandatory* segment and N_i^{seg} optional segments. While the mandatory segments of all tasks must finish by the respective deadline, executing optional segment allows refining the quality of output, thus higher *utility*. Evidently, executing optional segment of a task is optional but if executed provides higher utility. Let $G_{i,0}$ denote the mandatory segment of task T_i and $G_{i,k} \mid k \in [1, N_i^{seg}]$ be the k^{th} optional segment of T_i . Executing $G_{i,k}$ implies all previous segments of this task are also executed. Let $e_{i,k}$ denote the execution time for executing up to $k^{th} \mid k \in [0, N_i^{seg}]$ optional segment of T_i with f^{max} . Note, $e_{i,0}$ is the execution time to execute mandatory segment of task T_i with the highest frequency f^{max} .

Let $h = \text{lcm}\{p_i\}$ be the hyper-period. Then, the number of jobs of T_i in the hyper-period is $N_i^{job} = \frac{h}{p_i}$. We allow jobs of a task to execute with a different frequency. However, once a job starts executing it cannot execute with a different frequency [1].

Utility Model. We assume that the utility gained by execut-

ing optional segment is weighted, however remains the same for all jobs of a task. Furthermore, we assume that utility values are already adjusted and u_i denote the utility gained by per unit execution of optional segment of task T_i . In this work, we consider a *linear* utility function.

Energy Model. Following the common convention, we follow the following energy model.

The effective execution time to execute job $T_{i,j}$ up to k^{th} segment (and hence, all previous segments) with z^{th} frequency is given as

$$e_{i,j,k,z}^{eff} = \frac{1}{f_z} \sum_{q=0}^k e_{i,q} \quad (1)$$

Then, the corresponding energy consumption is

$$E_{i,j,k,z} = \alpha e_{i,j,k,z}^{eff} + \beta f_z^2 \sum_{q=0}^k e_{i,q} \quad (2)$$

Energy Budget. The total energy consumption should not exceed energy budget $\mathcal{B}$.

Problem Statement. The problem of utility aware scheduling of real-time task with energy budget is referred to USRT.

USRT problem is to compute a job to core, frequency and segment mapping $T_{i,j} \mapsto \langle P_x, f_z, k \rangle$ to optimize total utility in the hyper-period such that

- 1) timing constraint is satisfied for each job in the hyper-period that is to say, the mandatory segment of all jobs and selected optional segments must be executed before respective deadline as per EDF
- 2) total energy consumption does not exceed the energy budget $\mathcal{B}$
- 3) scheduling constraints are satisfied - no migration, no change in frequency, exclusivity (only one job can be executed by a processor at a time)

III. ILP FORMULATION V1

Let $V(ijkxz)$ be a binary variable which is 1 if job $T_{i,j}$ executes on processor P_x with the frequency f_z up to k^{th} segment such that $i \in [1, N^{tsk}]$, $j \in [1, N_i^{job}]$, $k \in [0, N_i^{seg}]$, $x \in [1, N^{prc}]$, $z \in [1, N^{freq}]$. Let $Y(ijkz)$ is an auxiliary variable defined as $Y(ijkz) = \sum_x V(ijkxz)$. Then, USRT is to compute $V(ijkxz)$ such that following *constraints* are satisfied.

[C1.] Each job executes with exactly one frequency and up to zero or more optional segments on a processor.

$$\sum_{k,x,z} V(ijkxz) = 1, \forall i, j \quad (3)$$

[C2.] Timing constraint is satisfied for each job. We use demand bound function DBF to determine feasibility.

Let $\mathcal{A} = \{r_{i,j}\}$ and $\mathcal{D} = \{d_{i,j}\}$ be the set of arrival time and deadline, respectively, of all jobs $(i \in [1, N^{tsk}], j \in [1, N_i^{job}])$ in the hyper-period. For $t_1 \in \mathcal{A}$, $t_2 \in \mathcal{D}$, let $\mathcal{S}(t_1, t_2)$ be the set of jobs such that $t_1 < t_2$ and $T_{i,j} \in \mathcal{S}(t_1, t_2)$ provided $t_1 \leq r_{i,j}$ and $t_2 \geq d_{i,j}$. Then, the demand bound during $[t_1, t_2]$ at processor P_x is given as

$$DBF(t_1, t_2, x) = \sum_{T_{i,j} \in \mathcal{S}(t_1, t_2)} \sum_{k,z} V(ijkxz) \times e_{i,j,k,z}^{eff} \quad (4)$$

Then, the task set is feasible if

$$DBF(t_1, t_2, x) \leq t_2 - t_1, \forall x, t_1 \in \mathcal{A}, t_2 \in \mathcal{D} \quad (5)$$

[C3.] Total energy consumption E^{total} does not exceed energy budget.

$$E^{total} \leq \mathcal{B} \quad (6)$$

where E^{total} is given as

$$E^{total} = \sum_{i,j,k,z} Y(ijkz) \times E_{i,j,k,z} \quad (7)$$

where $E_{i,j,k,z}$ is computed with Eq. (2).

Objective: The objective is optimize (maximize) sum of average weighted utility in the hyper-period accumulated by executing optional segments.

- We assume utility values are already adjusted to incorporate the weight.
- Let u_i denote the utility gained by per unit execution of optional segment of task T_i .

Utility gained by executing job $T_{i,j}$ is

$$U_{i,j}^{job} = u_i \times \sum_{k \in [1, N_i^{seg}], z} Y(ijkz) \times \sum_{q=1}^k e_{i,q} \quad (8)$$

Utility gained by task T_i over all jobs in the hyper-period

$$U_i^{tsk} = \sum_j U_{i,j}^{job} \quad (9)$$

The objective Φ is

$$\Phi = \sum_i U_i^{tsk} \quad (10)$$

IV. ILP FORMULATION V2

$SPS(\text{for mapping}) \rightarrow ILP(\text{for optional segment} + \text{freq})$

- First compute a mapping with SPS of jobs (mandatory segments only with maximum frequency). Let $P_{i,j}^* \in [1, N^{prc}]$ be the processor assigned to job $T_{i,j}$. Also, let $\mathcal{J}_x$ be the set of jobs mapped to processor P_x .
- For this mapping then determine the number of optional segments to execute and corresponding frequency for each job.

Let $Y(ijkz)$ is an auxiliary variable be a binary variable which is 1 if job $T_{i,j}$ executes with the frequency f_z up to k^{th} segment such that $i \in [1, N^{tsk}]$, $j \in [1, N_i^{job}]$, $k \in [0, N_i^{seg}]$, $z \in [1, N^{freq}]$. $T_{i,j}$ is scheduled on processor P_x as determined by SPS. Then, USRT is to compute $Y(ijkz)$ such that following *constraints* are satisfied.

[C1.] Each job executes with exactly one frequency and up to zero or more optional segments.

$$\sum_{k,z} Y(ijkz) = 1, \forall i, j \quad (11)$$

[C2.] Timing constraint is satisfied for each job. We use demand bound function DBF to determine feasibility. Specifically, let $\mathcal{A}_x$ and $\mathcal{D}_x$ be the set of all arrival time and deadline, respectively, of all jobs mapped to the processor P_x , i.e.,

$\mathcal{A}_x = \{r_{i,j}\}$ and $\mathcal{D}_x = \{d_{i,j}\}$ provided $P_{i,j}^* = P_x$. For a processor P_x and $t_1 \in \mathcal{A}_x, t_2 \in \mathcal{D}_x$, let $\mathcal{S}(t_1, t_2, x)$ be the set of jobs such that $t_1 < t_2$ and $T_{i,j} \in \mathcal{S}(t_1, t_2, x)$ provided $P_{i,j}^* = P_x, t_1 \leq r_{i,j}$ and $t_2 \geq d_{i,j}$. Then, the demand bound during $[t_1, t_2]$ at processor P_x is given as

$$DBF(t_1, t_2, x) = \sum_{T_{i,j} \in \mathcal{S}(t_1, t_2, x)} \sum_{k,z} Y(ijkz) \times e_{i,j,k,z}^{eff} \quad (12)$$

Then, the task set is feasible if

$$DBF(t_1, t_2, x) \leq t_2 - t_1, \forall x, t_1 \in \mathcal{A}_x, t_2 \in \mathcal{D}_x \quad (13)$$

[C3.] As it is in v1.

Objective: As it is in v1.

V. HEURISTIC

Decisions to make

- **Mapping** Obtain mapping with SPS.
 - **Ch1** Use this mapping as it is
 - **Ch2** Refine mapping as SPS is unaware of optional segment and utility. SPS gives a balanced mapping at fmax of mandatory segments only. Thus, the tasks with higher utility may be mapped together but then due to timing constraint most of the optional segments cannot be executed.
 - * **Goal** Rearrange jobs such that total utility density across all processor is balanced while ensuring that timing constraint remain satisfied

A. Refine Mapping

- SPS unaware of freq and optional seg
- If higher utility density jobs are mapped together then even if energy budget allows, timing constraint may prevent scheduling additional optional segment
- $\Rightarrow$ Adjust mapping to minimize max utility density
- Not brute-force/exponential
- Goal is not to find optimal mapping, just to minimize max utility density as much as possible
- Not allowing this in ILP v2

Steps

- **Step 1** Arrange job pairs $\mathcal{B} = \{(_, _), (_, _), \dots, (_, _)\}$ in the order of **increasing** difference of utility density
- It is desirable that jobs with lower difference are not together
- **Choice** Every pair must be considered **constant/bounded** number of times
- **Step 2.a** Linear loop over $\mathcal{B}$
 - Consider pair of jobs $p \in \mathcal{B}$
 - Swap if jobs are not on the same processor and utilization density difference between processors reduces
- **Step 2.b** One swap, one scan
 - (**Like SPS**), consider pair of processors $\langle P_1, P_2 \rangle$ with the highest utility density difference (say, P_1 with higher utility density)
 - Find first pair $p \in \mathcal{B}$ | higher utility density job of p is on P_1
 - Swap jobs of pair p
 - Update utility density

1) LS - Frequency, Optional Segment:

- 1) **Left shift** - shift all jobs to the left, each job start at $\max(\text{processor available time or finish time of previous job, job release time})$
 - With respect to any job, the time slack is in future now
 - Maintain left shift invariant, if inc freq of a job, all future jobs may need to be shifted to left. Also, when decrease optional segment.
 - Also, whenever increase optional segment or dec freq, right shift is needed
- 2) Compute energy slack $\Delta^e = \mathcal{B} - E^{total}$
- 3) (fmax and only mandatory segment)
- 4) (reduce freq first) Iterative one level scale down for all jobs until timing energy constraints are satisfied. That is to say, **Repeat** the following until one job's frequency is reduced in current iteration
 - For each job $\rightarrow$ Scale down its frequency by one level provided timing constraints for this and all jobs mapped with this job satisfies timing constraint and $\Delta^e > 0$
- 5) (optimize for the objective $\rightarrow$ schedule as many optional segments as possible)
 - For job J^* with the highest utility density - all jobs are scheduled with some freq and up to some optional segment - while satisfying timing and energy constraint do the best that is possible wrt J^*
 - a) **Ch1** Without impacting others
 - Incrementally increase optional segment of J^* while satisfying timing + energy constraints
 - $\rightarrow$ energy budget available but timing constraint violated $\rightarrow$ Incrementally increase frequency of J^* to schedule more optional segments
 - $\rightarrow$ time slack available but energy budget is not available $\rightarrow$ Incrementally decrease frequency of J^* to schedule more optional segments
 - $\rightarrow$ time slack and/or energy budget not available $\rightarrow$
 - b) **Ch2** Without impacting objective wrt others (optional segments of jobs other than J^* remains as it is (only jobs with overlapping activation window with J^* required to be considered)
 - (Time slack not avail, energy budget avail) Incrementally inc freq of others, schedule additional segments for J^* while satisfying timing+energy constraints
 - (Time slack avail, energy budget not) Incrementally dec freq of others, ...
 - c) **Ch3** (time and/or energy slack not available) Decrease optional segment of others (Only for jobs with lower utility density and for jobs that overlap activation window of J^*)

- Incrementally reduce optional segment of others
- **Ch1**
- **Ch2**

Another choice to make - Timing constraint is per processor once mapping is fixed. Energy constraint can be defined globally or per processor by distributing energy budget.

B. Heur1.0/Baseline

- 2-Phase
- Phase 1 - Obtain a mapping feasible with fmax with no optional segment
 - Consider tasks in the decreasing order of utilization (**experiment with others**), processors in the decreasing order of total utilization, map a task to first least utilized processor if feasible with UDF (simple utilization check is sufficient, no need of DBF yet)
- Phase 2 - Local search for frequency assignment and determining number of optional segments to execute
 - No migration, inc/dec frequency, inc/dec number of optional segment executed
 - Operations
 - 1) inc freq - energy consumption increases, effective execution time dec, by default deadline satisfied, no impact on obj [may be helpful in future, a bad greedy move]
 - 2) dec freq - energy consumption reduces, effective execution time inc+deadline violation possible, no impact on obj
 - 3) inc number of optional seg - energy consumption inc, eff exec time inc, deadline may be violated, obj improves
 - 4) dec number of optional seg - energy consumption reduces, eff exec time dec, by default deadline satisfied, obj reduces [may be helpful in future]
 - 5) inc freq + inc optional seg - energy consumption inc, effective execution time dec/inc, deadline satisfied/may violate, obj improves
 - 6) inc freq + dec optional seg - energy consumption inc/dec, effective execution time dec, deadline satisfied, obj dec
 - 7) dec freq + inc optional seg - energy consumption inc/dec, effective execution time inc, deadline satisfied/may violate, obj improves
 - 8) dec freq + dec optional seg - energy consumption inc, effective execution time dec/inc, deadline satisfied/violate, obj dec
 - Iteratively, in each iteration apply operations O1-O8, select the best solution if obj improves, probabilistically accept a bad solution
 - * Prob to accept a bad solution decreases geometric scaling, constant factor

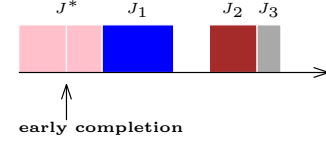


Fig. 2. Offline Information (To be used online)

At the runtime, need to decide **when** and **where** to use time and energy slack.

Once a job completes early, use its time slack and energy slack to execute future jobs while selecting one of the following options.

- 1) Release the next job, save time slack Δ^t and energy slack Δ^e for future.
- 2) Use time slack Δ^t and energy slack Δ^e to execute additional optional segments.
- 3) Use time slack Δ^t to decrease frequency, increases energy slack Δ^e to be used later.
- 4) Use energy slack Δ^e to increase frequency, Δ^t to be used in (near) future.

Distribution of Δ^t and Δ^e : Based on offline computed information with a DP based approach. Specifically, once J^* completes in Figure 2, need to decide online how to use Δ^t and Δ^e among jobs $\{J_1, J_2, J_3\}$.

Steps. Consider jobs from right to left, when job J_1 is under consideration assume that is left shifted while subsequent jobs are right shifted. Then, apply DP like approach to compute how to distribute $(\Delta^t = 0, \Delta^e = 0), (\Delta^t = 1, \Delta^e = 0), (\Delta^t = 0, \Delta^e = 1), (\Delta^t = 1, \Delta^e = 1), \dots$. This requires reusing the solution computed for subsequent job. Thus, while deciding how to distribute (Δ^t, Δ^e) to J_1 (and subsequent jobs) it is the **best** of (i). (Δ^t, Δ^e) to J_1 and $(\Delta^t = 0, \Delta^e = 0)$ to J_2 , (ii). $(\Delta^t - 1, \Delta^e)$ to J_1 and $(\Delta^t = 1, \Delta^e = 0)$ to J_2 , (iii). $(\Delta^t, \Delta^e - 1)$ to J_1 and $(\Delta^t = 0, \Delta^e = 1)$ to J_2 , and so on.

Theoretically, $\Delta^e \leq B$ and $\Delta^t < d_1$. **Better bounds** must be computed from schedule.

Questions to answer

- How to make schedule computation fast and efficient offline in terms of time and space?
- How to use it online? As it is or additional computation?
- Δ^e is shared, what if two early completions?
 - Approximate - consider the job which has more higher density in the future. This can be computed offline too.

Improvements

- Better bounds
- Aggregate states
- Lazy DP

VI. ONLINE

Utilize time and energy slack online due to early completion of a job (of offline scheduled mandatory+optional segments).

VII. TEMP

VIII. LITERATURE

Online Energy + Time Slack - not addressed in general

Optimal IC Task Mapping to Maximize QoS on Heterogeneous Multicore Systems - No DVFS [Borrow literature from this paper]

Prepare: Power-Aware Approximate Real-time Task Scheduling for Energy-Adaptive QoS Maximization - Online also, heterogeneous, dependent tasks. But one task graph, tasks have common deadline, no jobs. Non-preemptive. Do not consider early completion, rather cache miss stall

Voltage-scaling scheduling for periodic real-time tasks in reward maximization

Optimal reward-based scheduling for periodic real-time tasks

REFERENCES

- [1] N. Kumar, J. Mayank, and A. Mondal, "Reliability aware energy optimized scheduling of non-preemptive periodic real-time tasks on heterogeneous multiprocessor system," *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 4, pp. 871–885, 2019.